

真实软件开发中主动式编程助手的实证研究

Lehui Li^{*,1,2}, Ruixuan Jia^{*,1}, Guo-Ye Yang², Jia Li^{†,1}

¹College of AI, Tsinghua University, China

²Fitten Tech Co., Ltd., China

* 共同一作 † 通讯作者

Abstract—基于大语言模型 (LLM) 的编程助手能力不断增强, 但大多数系统仍然停留在被动式交互范式中, 只有在开发者明确发出指令后才会提供帮助。主动式编程助手则试图从集成开发环境 (IDE) 的交互轨迹和仓库上下文中预测开发者的隐式意图, 从而降低书写指令的认知负担并提升开发效率。然而, 由于缺乏大规模真实世界开发者行为数据, 现有研究高度依赖 LLM 生成的模拟数据, 而这些数据与真实数据之间到底有多接近仍不清楚。本文通过大规模真实数据采集研究这一从模拟到现实的落差。我们从 1,246 名有经验的工业界开发者处连续三天采集 IDE 交互轨迹, 并为每条真实轨迹构造配对的 LLM 模拟轨迹, 以便进行可控对比。分析结果表明, LLM 模拟轨迹在行为多样性、时间结构和探索性行为方面都与真实轨迹存在显著差异。基于这些真实轨迹, 我们构建了面向真实开发场景中主动意图预测的基准 ProCodeBench。在代表性的 LLM、检索增强方法以及基于 Agent 的基线上的实验表明, 现有方法在真实 IDE 轨迹下距离可靠仍有明显差距, 这说明基于模拟的评测可能高估其真实意图预测能力。最后, 我们的训练研究表明, 仅依赖 LLM 模拟数据无法替代真实数据, 但如果先用模拟数据再用真实数据微调, 则能带来性能提升。这些发现强调了真实开发者行为数据对于评测和训练主动式编程助手的重要性, 同时也揭示了 LLM 模拟数据所能发挥的互补作用。

Index Terms—主动式代码辅助, AI4SE, benchmark, real-world data

I. 引言

近年来, 大语言模型 (LLM) 的进展显著提升了它们在代码生成 [1, 2]、测试生成 [3] 等软件工程任务上的表现, 也使基于 LLM 的编程助手越来越多地被集成进现代软件开发流程中 [4–8]。它们的能力已经从早期的代码补全和重构 [9, 10], 扩展到支持多轮交互和自主执行任务的最新编程 Agent [11]。尽管如此, 多数编程助手仍遵循被动式交互范式, 只有在开发者明确发出指令后才会提供帮助 [12, 13]。这种设计限制了它们在真实开发场景中的实用性。一方面, 开发者在编码过程中必须持续构造细致的指令, 这会带来额外的认知负担 [14–16]。更重要的是, 软件工程任务往往相当复杂, 开发者未必总能清晰表达自己的开发意图 [14]。

为了解决这些限制, 近期研究提出了主动式编程助手 [12, 13, 17]。如 Figure 1 所示, 与等待显式指令的被动式编程助手不同, 主动式编程助手会从 IDE 交互轨迹和仓库上下文中推断开发者的潜在意图, 并据此提供对应的辅助建议。先前用户研究表明, 只要能够识别开发者 IDE 轨迹中蕴含的意图, 主动式编程助手平均可将任务表现提升 12%–18%, 同时还会给大多数参与者带来明显的使用体验改善 [14, 15]。

然而, 主动式编程助手仍然缺乏大规模真实世界开发者行为数据 [12, 13, 17]。与传统软件工程任务不同, 主动式代码辅助需要在真实开发过程中持续记录开发者的 IDE 交互轨迹 [17], 这使得数据采集成本高昂, 并受到严格的隐私约束。因此, 现有研究大多依赖 LLM 生成的模拟数据 [12, 13, 17, 18]。它们通常利用基于 LLM 的用户 Agent 合成交互轨迹, 并生成相应的意图标签, 再据此训练和评估主动式编程助手的意图预测能力。

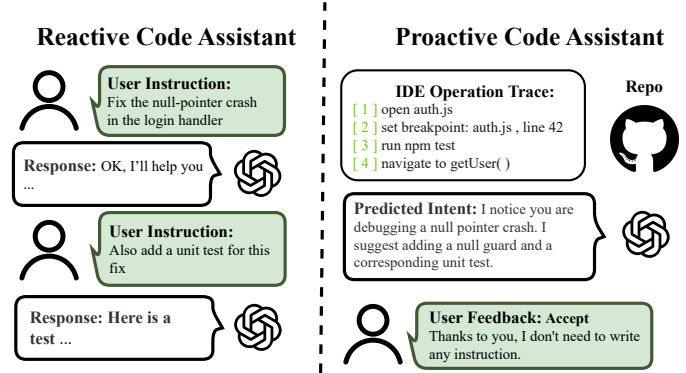


Fig. 1: 被动式与主动式编程助手的对比。被动式编程助手需要开发者在每次交互时显式给出指令, 而主动式编程助手则根据 IDE 交互轨迹预测开发者的潜在意图, 从而减少显式请求的需要。

尽管 LLM 生成的模拟数据推动了主动式编程助手的发展, 但这些数据是否真正反映了开发者在真实开发场景中的行为仍不清楚。这引出了我们的第一个研究问题:

RQ1: LLM 生成的模拟数据能否真实刻画开发者在现实世界中的行为及其背后的意图?

为回答这一问题, 我们通过 Visual Studio Code (VS Code) 扩展采集了超过 400 万条真实 IDE 交互轨迹。数据覆盖 1,246 名志愿者连续三天的开发过程, 涉及前端、后端、数据库和算法工程等代表性开发场景。对于每条真实轨迹, 我们进一步合成一条配对的 LLM 模拟轨迹, 从而实现真实数据与模拟数据之间的可控比较。差异分析清楚地揭示出模拟到现实之间的鸿沟: 真实轨迹具有更强的行为多样性、更细粒度的操作以及更复杂的时间模式, 也包含更丰富但更嘈杂的过程级信息; 相比之下, LLM 模拟轨迹虽然更易生成, 却往往遵循过度简化的行为模式, 难以逼近真实开发过程。

由于模拟数据与真实数据存在显著差异, 基于模拟基准的评测可能会高估模型主动预测用户意图的能力。因此我们进一步考察:

RQ2: 现有主动式编程助手在真实世界数据上的实际表现究竟如何?

为回答 RQ2, 我们基于采集到的真实 IDE 交互轨迹构建了 ProCodeBench。由于开发者意图隐含在连续的 IDE 操作中, 我们通过一条标注流水线将原始轨迹转换为标准化的意图预测样本。随后, 我们对 13 个具有代表性的强基线进行了系统

评测, 其中包括 7 个当前主流 LLM (如 GPT-5.4、Claude Sonnet 4.6 和 Gemini 3.1 Pro)、4 个检索增强 LLM (如 RepoCoder 和 RepoGraph) [19, 20], 以及 2 个基于 LLM 的 Agent (SWE-Agent 和 A-RAG) [11, 21]。结果得到三点结论: ① 现有基线在真实 IDE 轨迹上的开发者意图预测仍然困难, 性能显著低于模拟基线报告的结果 [17, 18]; ② 仓库级代码上下文能够稳定提升不同骨干模型上的意图预测效果, 说明仓库信息有助于澄清观察到的 IDE 操作背后的目的; ③ 基于 LLM 的 Agent 通过多轮工具使用取得了最佳结果, 但如何高效且有效地利用仓库级上下文依然是未解决的挑战。

RQ2 中观察到的真实世界性能不足进一步引出了一个问题:

RQ3: 使用模拟数据或真实数据训练, 能否提升主动意图预测能力?

为回答 RQ3, 我们在统一设置下比较了基于真实数据、LLM 模拟数据以及混合数据训练方案得到的模型。结果表明, 仅使用 LLM 模拟数据训练并不能很好迁移到真实开发场景; 但如果先用 LLM 模拟数据进行初始化, 再使用真实数据微调, 真实世界性能就能得到提升。这表明真实数据与 LLM 模拟数据并不是可互换的替代品, 而是能够共同提升主动意图预测能力的互补数据源。本文的主要贡献如下:

- ① 我们提出了首个面向主动式代码辅助的大规模真实世界数据集。该数据集包含来自 1,246 名志愿者、跨越代表性开发场景、连续三天的 IDE 交互轨迹。通过将每条真实轨迹与一条 LLM 模拟轨迹配对, 我们揭示了开发者行为中清晰的“模拟到现实”差距, 包括行为多样性、操作粒度、时间模式和过程级噪声等方面的差异。
- ② 我们构建了 **ProCodeBench**, 这是首个在真实开发场景下评测主动式代码辅助能力的基准。我们通过标注流水线将原始 IDE 交互轨迹转化为标准化意图预测样本, 并提供统一评测协议。对主流 LLM、检索增强 LLM 以及基于 LLM 的 Agent 的实验表明, 现有模型在真实世界主动意图预测上依然表现吃力。
- ③ 我们分析了真实数据、LLM 模拟数据以及混合训练方案对模型训练的作用。结果表明, 仅使用 LLM 模拟数据无法很好泛化到真实开发场景; 但若在真实数据微调前先用其进行初始化, 则可以提升最终性能。这说明 LLM 模拟数据与真实数据扮演的是互补而非可互换的角色。

II. 相关工作

基于 LLM 的编程助手. 基于 LLM 的编程助手在软件工程任务上发展迅速, 覆盖了代码补全与生成 [9, 10, 22]、程序修复 [23]、自动化测试生成 [3] 以及仓库级代码理解 [24] 等方向。SWE-Agent [11] 进一步引入了带工具使用能力的多轮推理, 使模型能够自主浏览代码库、检索上下文并执行修复。与此同时, Cursor、GitHub Copilot 和 Windsurf 等商业编程助手也已经深度集成进 IDE, 成为开发流程的一部分。尽管如此, 现有系统在本质上仍是被动式的, 只会在收到显式用户请求后提供帮助, 无法在用户意图尚未表达清楚之前主动介入。本文关注的主动式代码辅助正是为了解决这一限制 [12, 13, 17]。

主动式辅助. 近期研究开始探索主动式辅助 [12, 13, 17, 18, 25], 其核心思想是根据行为序列和上下文预测用户的潜在意图, 并在没有显式提问的情况下主动给出建议。ProActiveAgent [17] 首次形式化了主动式辅助任务, 并通过 LLM

模拟构建了 ProActiveBench, 覆盖编程、写作等多个场景。ProActiveSim [18] 则将模拟框架扩展到日常生活场景。CodingGenie [13] 在 VS Code 中实现了一个主动式编程助手原型, 并通过用户研究评估其交互设计, 但并未进行系统化基准测试。上述工作的共同限制在于, 它们的训练和评测数据完全依赖于 LLM 生成的 IDE 模拟轨迹 [17, 18, 26]。值得注意的是, 一些同期工作 [27, 28] 通过纳入用户设备上捕获的真实截图来提升数据真实性。然而, 截图只能反映界面的瞬时状态, 缺乏细粒度操作信号, 如编辑增量、终端输出和光标移动, 也缺少仓库级代码上下文。此外, 这类方法主要面向通用 GUI 场景, 并不适合代码开发中的主动式辅助。相比之下, 本文提出了首个源自真实 IDE 交互轨迹的主动式代码辅助基准。

面向真实软件开发的基准与数据集. 软件工程研究已经建立了大量基准 [29, 30]。HumanEval [1] 和 MBPP [31] 评测函数级代码生成, SWE-bench [32] 将评测扩展到仓库级, CodeSearchNet [33] 与 DevBench [34] 则为代码理解和维护提供了大规模数据。然而, 这些基准都以静态代码或自然语言描述作为输入 [1, 31–34]。主动式代码辅助则不同, 它处理的是开发者在开发过程中产生的动态 IDE 交互轨迹, 而这类数据模态在现有软件工程基准中是缺失的。ProCodeBench 通过将真实 IDE 交互轨迹纳入主动式代码辅助的训练与评测, 填补了这一空白。

III. 任务定义

不同于依赖显式指令的被动式编程助手, 主动式编程助手旨在根据开发者的 IDE 交互轨迹和仓库上下文预测其隐式意图, 并在开发者发出明确请求之前提供帮助。形式化地说, 我们将 IDE 操作定义为开发过程中记录下来的用户动作, 例如编辑代码、切换视图、选择代码或执行终端命令。每个操作表示为一个四元组 $o_i = (p_i, g_i, c_i, t_i)$, 其中 p_i 表示操作类型, g_i 表示目标实体 (如文件、函数或选中代码区域), c_i 表示操作内容, t_i 表示时间戳。由 n 个连续 IDE 操作构成的序列记为 IDE 交互轨迹 $\mathcal{O} = \{o_1, o_2, \dots, o_n\}$ 。给定该轨迹 $\mathcal{O}$ 与仓库级代码上下文 $\mathcal{C}$, 任务目标是预测开发者意图:

$$\text{Intent} = f_{\theta}(\mathcal{O}, \mathcal{C}),$$

其中 f_{θ} 是参数为 θ 的主动式编程助手, 输出为一段自然语言意图描述。Figure 2 给出了一个代表性例子: 当系统观察到开发者先定义了 `retry_with_backoff` 装饰器, 随后又查看了多个调用 API 的模块时, 它应当预测出开发者的意图是将该装饰器应用到数据流层的相关函数上。

IV. 研究方法

如 Figure 3 所示, 我们的研究方法由三个阶段构成, 每个阶段对应一个研究问题。**阶段 1: 数据采集与差异分析** 从资深工程师志愿者处收集真实 IDE 交互轨迹, 并为每条轨迹构造一个 LLM 模拟配对样本, 随后量化这两类数据之间的分布差异。**阶段 2: 基准构建** 通过意图标注流水线, 将采集到的真实 IDE 交互轨迹重构为一个面向主动式代码辅助的标准化基准, 并配备统一的评测协议和代表性基线。**阶段 3: 训练分析** 则在该基准上通过不同训练方案的可控比较, 考察真实数据和 LLM 模拟数据在模型训练中的各自作用。

ProCodeBench 中的一个代表性样例

输入 — IDE 交互轨迹 $\mathcal{O}$ (开发者正在处理一个 Python 项目)

```
01 COPY "handle API call error and retry; review cli/*.py
and tradingagents/*.py"
02 VIEW open dataflows/utils.py
03 EDIT in utils.py: add import time, functools; de-
fine retry_with_backoff(retries, backoff) dec-
orator
04 VIEW open dataflows/alpha_vantage_common.py (con-
tains API functions)
05 EDIT in alpha_vantage_common.py: add import time
06 VIEW switch back to utils.py to inspect the decorator
07 VIEW switch back to alpha_vantage_common.py
```

输出 — 预测意图 $\text{Intent} = f_{\theta}(\mathcal{O}, \mathcal{C})$

将新定义的 `retry_with_backoff` 装饰器应用到数据流模块中调用 API 的相关函数上。

Fig. 2: ProCodeBench 中的一个代表性样例。该 IDE 交互轨迹先复制了一条与 API 重试相关的说明，又在 `utils.py` 中定义了 `retry_with_backoff` 装饰器，随后打开多个调用 API 的模块，这些行为自然指向一个潜在意图：将新的装饰器应用到数据流层中。操作类型包括 **COPY**、**VIEW** 和 **EDIT**。

A. 阶段 1: 数据采集与差异分析

为了回答 RQ1 (LLM 生成的模拟数据能否真实刻画开发者行为及其底层意图?)，我们需要从志愿者处获取大规模、真实的 IDE 交互轨迹，并同时获得与之配对的 LLM 模拟轨迹，以便进行可控比较。

真实数据采集. 为了收集真实开发者行为数据，我们首先开发了一个 VS Code 扩展，用于记录广泛的 IDE 操作。如 Table I 所示，该扩展捕获了 8 类最常见的用户与 IDE 交互操作，包括编辑、复制粘贴、视图切换、光标选择、终端执行和调试等。特别地，它还记录了 AI 辅助开发行为，包括通过 `code completion` 接受的代码补全，以及通过 `agent request` 发给编程 Agent 的自然语言请求。每条操作记录都以结构化载荷的带时间戳 JSON 形式存储。例如，一次 `edit` 操作会记录被编辑文件的路径、插入与删除的文本片段，以及编辑前后的周边代码上下文。

此外，我们招募了 1,246 名有经验的工业界开发者作为志愿者，覆盖五类主要开发场景：后端开发 (412 人, 33.1%)、前端开发 (287 人, 23.0%)、全栈开发 (208 人, 16.7%)、算法工程 (183 人, 14.7%) 和数据库开发 (156 人, 12.5%)，具体统计见 Table II。在连续三天的采集期间，每位志愿者都在自己持续维护的工业项目上启用该 VS Code 扩展进行开发。所有志愿者在完成采集后均获得经济报酬。最终，我们从真实 IDE 交互轨迹中收集了约 **463 万条操作事件**。

LLM 模拟数据构造. 为了构造配对的合成数据，我们遵循 ProActiveAgent [17] 的流程。对于每条真实 IDE 交互轨迹，我们生成一条对应的 LLM 模拟轨迹。为保证比较公平，模拟器会获得与志愿者一致的画像信息，包括工作背景、开发经验和主要技术栈。我们还约束每条 LLM 模拟轨迹在长度上与其对应的真实轨迹保持一致，并且只使用相同的 8 类操作类型。

差异分析. 基于成对的真实 IDE 交互轨迹和 LLM 模拟轨迹，我们从三个角度分析它们的分布差异：行为多样性、时间模

TABLE I: 我们的 VS Code 扩展捕获的 8 类 IDE 操作。每条操作记录都以带时间戳的 JSON 形式存储，并包含表中列出的字段。

Operation type	Captured information
<code>edit</code>	File path, the inserted and deleted text segments, and the surrounding code context with line range
<code>copy/paste</code>	Copied or pasted text content from the system clipboard
<code>view switching</code>	File path, the viewport line range, and the actual code content visible within the viewport
<code>cursor_selection</code>	File path, the cursor position or selected text span, and the surrounding code context with a cursor marker
<code>terminal_execution</code>	Shell command line, exit code, execution duration, and the captured terminal output
<code>debug</code>	Debug session identifier, output category (e.g., <code>stdout/stderr</code>), and the captured debug output
<code>code_completion</code>	File path, line number, the accepted completion text, and the surrounding code context
<code>agent_request</code>	The natural-language request issued by the developer to the coding agent

式以及噪声模式。行为多样性 通过操作类型的频率分布来度量，用以判断 LLM 模拟轨迹是否覆盖了真实开发过程中的长尾 IDE 操作。时间模式 则通过操作间时间间隔和操作类型转移矩阵来刻画，用于揭示模拟数据是否复现了真实编码过程中的多尺度时间结构以及频繁的操作切换。噪声模式 指的是那些与最终意图仅弱相关的操作，例如探索式文件浏览和冗余导航。我们还通过一个代表性案例来展示这些噪声模式在真实轨迹和 LLM 模拟轨迹中的不同体现。

B. 阶段 2: 基准构建

为回答 RQ2 (现有主动式编程助手在真实世界数据上表现如何?)，我们基于采集到的真实 IDE 交互轨迹构建了一个标准化评测基准。与 LLM 模拟数据不同，真实轨迹并不附带显式意图标签。更进一步地，开发者意图仅仅是隐含在连续 IDE 操作之中的，开发者可能会在不同意图之间切换，例如重构某个函数或调试某个异常，而这些意图之间并不存在清晰边界。因此，核心挑战在于如何将连续 IDE 交互轨迹转换为可评测的样本。为此，我们设计了一条由意图识别、意图筛选和人工复核组成的三步标注流水线 (Figure 4)。在此基础上，我们建立了统一的评测协议和一组有代表性的基线。

步骤 1: 意图识别. 为了从连续 IDE 交互轨迹中识别开发者意图，我们采用滑动窗口策略。每个窗口包含 N 个连续操作，由 LLM 从中识别潜在意图、定位其起止操作，并为每个意图赋予初始的自然语言描述。我们将 N 设为 50，以在识别质量和 LLM 标注成本之间取得平衡。

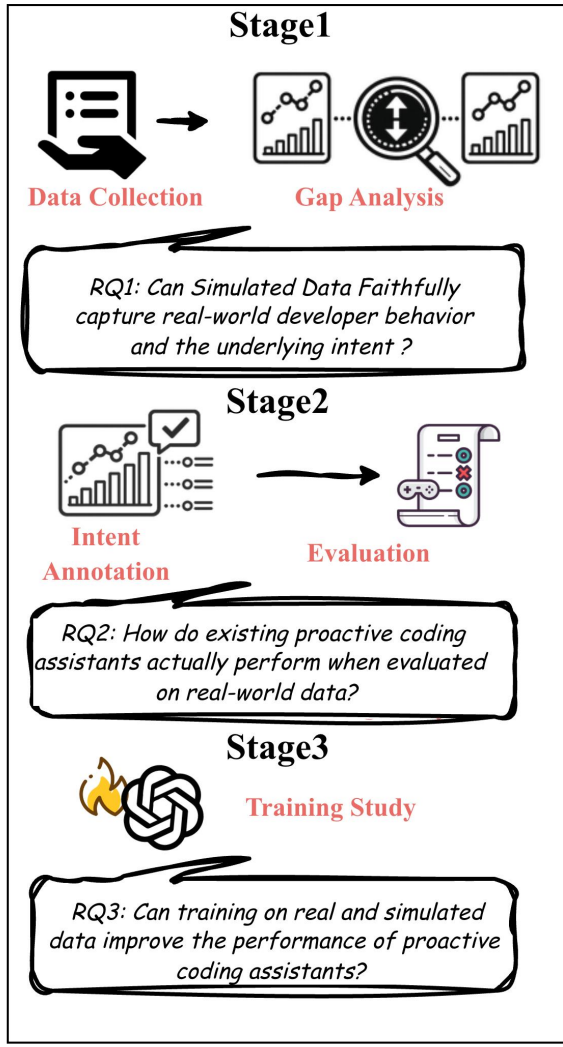


Fig. 3: 研究方法总览。阶段 1 收集成对的真实 IDE 轨迹与 LLM 模拟 IDE 轨迹，用于差异分析；阶段 2 通过对真实轨迹进行意图标注构建 ProCodeBench；阶段 3 比较基于真实数据、LLM 模拟数据以及混合数据训练方案的不同效果。

步骤 2：意图筛选。为了筛出值得评测的意图片段，我们对步骤 1 识别出的候选意图进一步筛选。由于开发者并不会显式指出哪些意图最值得获得辅助，我们使用可观察的行为信号作为代理。具体而言，我们采用两步筛选策略：启发式筛选保留那些包含显著代码编辑或显式 AI 助手请求的候选意图，因为它们通常对应可能受益于辅助的复杂开发任务；语义筛选则借助 LLM 判断每个保留片段是否连贯，并与其意图描述保持一致。

步骤 3：人工复核。为了保证标注质量，我们在自动识别与筛选之后增加了人工复核环节。自动化步骤可能带来两类错误：一类是保留下来的候选样本意图描述不准确，另一类是本应保留的有效候选被错误筛掉。两位领域专家独立检查这些候选样本，修正错误的意图描述，并从被过滤掉的集合中恢复有效样本。

数据集划分。经过三步标注流水线后，我们得到了 5,492 个有效评测样本。为避免时间泄漏，我们按时间顺序将样本划分

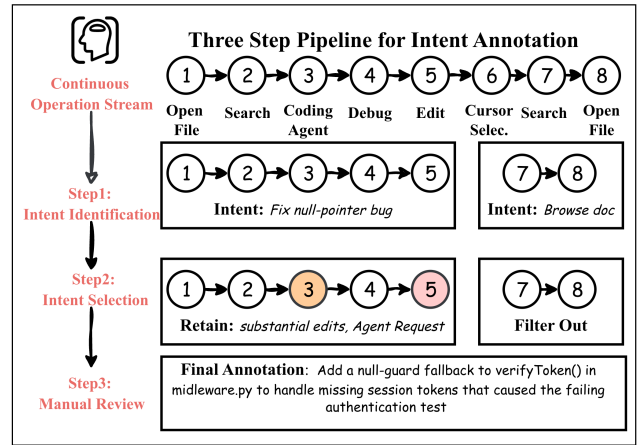


Fig. 4: 将连续真实 IDE 交互轨迹转化为标准化意图标注评测样本的三步意图标注流水线。

TABLE II: ProCodeBench 数据集统计。我们连续跟踪了 1,246 名志愿者 3 天，共得到 5,492 个标注样本。

Data split			
	Samples	Ratio	
Train	3,576	65.1%	
Validation	1,142	20.8%	
Test	774	14.1%	
Developer domains			
	Volunteers	Ratio	
Backend	412	33.1%	
Frontend	287	23.0%	
Full-stack	208	16.7%	
Algorithm	183	14.7%	
Database	156	12.5%	

为训练集、验证集和测试集，分别包含 3,576、1,142 和 774 个样本（见 Table II）。每个评测样本以前序 IDE 交互轨迹作为输入，以对应的自然语言意图描述作为输出。

评测协议。为了评估主动式编程助手的意图预测能力，我们将模型生成的意图与开发者的真实意图进行比较。由于开发者意图以自然语言描述表示，简单的精确匹配不足以判断语义正确性。因此，我们采用 **LLM-as-a-Judge** 的评测策略 [35]。对于每个样本，一个独立的 LLM 评审会同时接收模型生成的意图描述和开发者真实意图，并判断二者在语义上是否等价。

与完全依赖 LLM 判定的先前工作不同，ProCodeBench 提供了开发者真实意图作为真值，这有助于减轻 LLM 评审器可能带来的偏差 [36]。我们使用 **Pass@K** 作为主要指标：对于每个样本，模型独立生成 K 个意图描述，只要其中至少有一个被判定与开发者真实意图语义等价，就认为预测正确。

基线方法。我们在 ProCodeBench 上比较如下基线方法。

LLMs.为了评估当前前沿 LLM 的意图预测能力，我们从近期通用与软件工程基准中选取了若干广泛使用的模型，包括 DeepSeek-V3.2、GLM-5 [37]、MiniMax-M2.5、Qwen3.5-

397B [38]、GPT-5.4、Claude Sonnet 4.6 和 Gemini 3.1 Pro。这些模型以开发者的 IDE 交互轨迹为输入，并直接预测开发者意图。

Retrieval-Augmented LLMs. 为了评估模型如何利用仓库级代码上下文，我们纳入了基于 Retrieval-Augmented Generation (RAG) 的方法。非图结构方法，如 RepoCoder [19] 和 CodeRAG [39]，通过基于文本或向量嵌入的检索方式获取相关代码片段；图结构方法，如 GraphCoder [40] 和 RepoGraph [20]，则进一步通过静态代码关系（如调用依赖、导入关系和符号级链接）纳入仓库结构信息。

LLM-based Agents. 我们还评估了基于 LLM 的 Agent，包括 SWE-Agent [11] 和 A-RAG [21]。与只进行单次检索不同，这些 Agent 会通过多轮工具交互来获取仓库上下文，例如浏览文件、搜索代码和检查符号，随后再给出意图预测。值得注意的是，先前的主动式编程助手可以被视为对 LLM Agent 的扩展，只是额外具备了主动预测用户意图的能力。因此，我们 not 将先前主动式编程助手单独作为一个独立基线类别。

C. 阶段 3: 训练分析

为了回答 RQ3（使用模拟数据或真实数据训练能否提升主动意图预测？），我们比较了分别基于单一数据源训练，以及联合使用两类数据训练的模型表现。

训练方案. 我们比较三种训练方案，以研究真实数据和 LLM 模拟数据在模型训练中的作用。**+Real** 表示在真实训练集上微调骨干模型，**+Sim.** 表示在配对的 LLM 模拟训练集上微调，**+Sim.→Real** 则表示一种混合训练方案：模型先在 LLM 模拟数据上训练，再进一步在真实数据上微调。所有方案都在同一套真实验证集和测试集上评测。

骨干模型. 我们使用能够在 8×A800-80GB GPU 计算预算下完成训练的开源模型进行微调实验。为了在相近参数规模上比较不同模型家族的训练效果，我们选取了三种 LLM: **Qwen-3-8B** [41]、**GLM-4-9B** [42] 和 **LLaMA-3-8B** [43]。

V. 结果

本节围绕三个研究问题报告实验结果。

A. RQ1: 真实 IDE 轨迹与 LLM 模拟 IDE 轨迹之间的分布差异

我们希望揭示真实开发数据与模拟开发数据之间的分布差异，这也是 ProCodeBench 的核心动机之一。我们从三个互补视角分析这一差异：行为多样性、时间模式和噪声模式。

行为多样性. 如 Figure 5 所示，真实轨迹与 LLM 模拟轨迹在操作类型分布上存在显著差异。更具体地说，LLM 模拟轨迹集中在少量操作类型上，主要是代码编辑和导航，而真实轨迹则覆盖了更广泛的 IDE 操作。因此，一些在真实开发过程中经常出现的操作类型，包括 AI 辅助操作，在模拟数据中明显更少见。尤其值得注意的是，真实数据呈现出一个鲜明特征：占比最高的并不是代码编辑，而是 **cursor selection** (约 40%) 和 **view switching** (约 13%)。这说明开发者在真正动手修改代码之前，会花大量时间阅读、检查和导航代码。综合来看，这些观察表明，基于 LLM 的模拟器往往只生成有限的操作类型集合，而忽略了真实开发场景中的操作多样性。

时间模式. 为了揭示真实轨迹与 LLM 模拟轨迹在时间模式上的差异，我们同时分析了操作间时间间隔与操作类型转移。在操作间隔方面，真实轨迹呈现出明显的双峰分布，一个峰值接近 0.1s，另一个位于数十秒量级 (Figure 6)。这一模式反映了

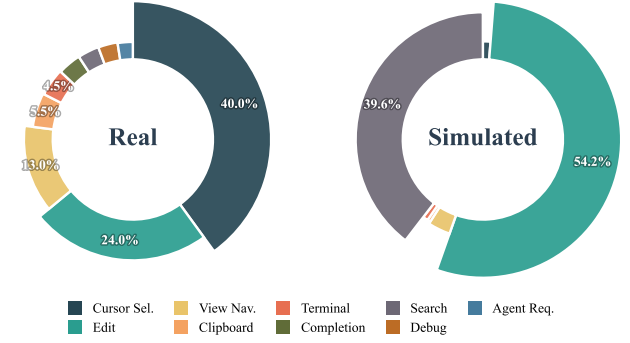


Fig. 5: 操作类型频率分布。真实世界数据覆盖了更广泛的操作类型，并呈现明显的长尾模式；而 LLM 模拟数据则集中在少数高频类别上。

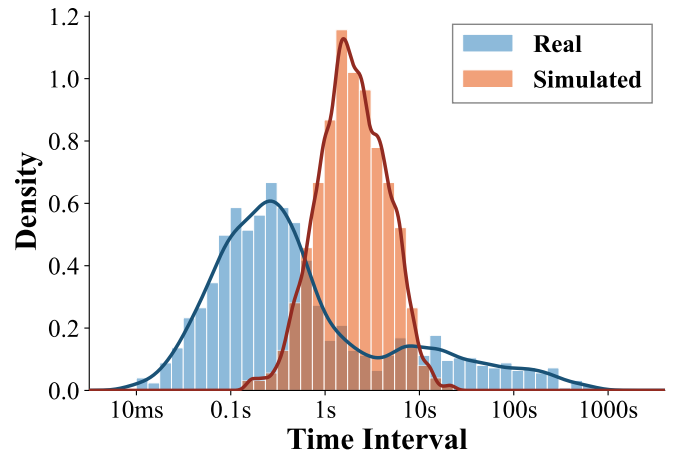


Fig. 6: 操作间时间间隔分布。真实世界数据呈现双峰模式，峰值分别出现在约 0.1 秒和数十秒附近；而 LLM 模拟数据则表现为一个以约 1 秒为中心的单峰分布。

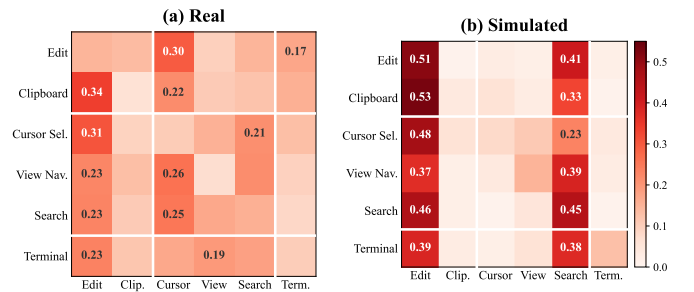


Fig. 7: 真实世界数据与 LLM 模拟数据中的操作类型转移模式。真实轨迹包含更丰富的跨操作类型转移，而 LLM 模拟轨迹则集中在少量高频转移上。

真实开发中的多尺度时间结构，即短暂密集的操作与较长时间的停顿交替出现。相比之下，LLM 模拟轨迹呈现以 1s 左右为中心的单峰分布，说明其时间结构更规则。在操作类型转移方面，真实轨迹分布在大量不同的操作对上 (Figure 7)，表明开发者会在多种开发活动之间频繁切换；而 LLM 模拟轨迹则集中

案例研究：真实 IDE 轨迹与 LLM 模拟 IDE 轨迹中的噪声模式

真实 IDE 交互轨迹（意图：修复登录处理器中的空指针崩溃）

[1] open auth.js	✓
[2] navigate to handleLogin()	✓
[3] set breakpoint auth.js:42	✓
[4] run npm run dev	✓
[5] open config.js	✗ redundant
[6] open README.md	✗ redundant
[7] back to auth.js	✗ redundant
[8] navigate to userService.js → getUser()	✓
[9] back to auth.js	✗ redundant
[10] edit handleLogin(): add null guard	✓
[11] undo last edit	✗ redundant
[12] edit handleLogin(): add guard with fallback	✓
[13] run npm test	✓
[14] select test output: 1 failed	✓
[15] edit handleLogin(): fix edge case	✓
[16] run npm test	✓

LLM 生成的模拟 IDE 交互轨迹（意图：为 API 请求增加重试逻辑）

[1] open api.js	✓
[2] navigate to fetchData()	✓
[3] edit fetchData(): add retry wrapper	✓
[4] edit fetchData(): add max retry config	✓
[5] open api.test.js	✓
[6] edit api.test.js: add retry test case	✓
[7] run npm test	✓

Fig. 8: 代表性噪声模式。真实 IDE 交互轨迹中包含探索式浏览（[3--4]）、冗余导航（[5]）以及被撤销的编辑（[7]）；而 LLM 生成的模拟 IDE 轨迹则呈现近乎线性的过程，几乎没有试错行为。

在少数相邻转移上，例如 Edit→Edit 和 Edit→Navigation。综合来看，这些结果说明基于 LLM 的模拟器并未充分复现真实编码过程中的时间结构与操作切换模式。

噪声模式。 真实轨迹中包含许多与最终意图仅弱相关的操作，因为开发者在探索和不断修正方案的过程中会频繁调整自己的动作。正如 Figure 8 所示，这些操作可能包括无关浏览、重复导航或被撤销的编辑。尽管它们引入了噪声，但也真实反映了开发过程中的不确定性和非线性。相比之下，LLM 模拟轨迹很少出现类似的探索性或纠错性行为，而更倾向于生成简化后的动作序列。这个对比说明，真实轨迹中包含人类探索行为所带来的噪声，而 LLM 模拟轨迹则带有明显的过度简化偏差。

关键发现 ①： LLM 生成的模拟轨迹无法真实刻画开发者在现实中的行为。与真实 IDE 交互轨迹相比，模拟轨迹的行为多样性更低，时间模式更规则，并且遗漏了真实编码过程中大量探索性行为。

B. RQ2: 现有主动式编程助手在真实世界数据上的表现

LLM 结果。 如 Table III 所示，所有 LLM 在真实测试集上的表现都相当有限，所有模型的 Pass@1 都低于 14%。即便是最强模型 Claude Sonnet 4.6，也仅达到 13.57%。这表明当前前沿 LLM 从真实 IDE 交互轨迹中预测开发者意图的能力仍然

TABLE III: ProCodeBench (759 个样本) 上的 LLM Pass@K 准确率 (%)。红色表示最佳，蓝色下划线表示次优。

Model	Pass@1	Pass@3	Pass@5
Claude-Sonnet-4-6	13.57	21.61	24.37
Gemini-3.1-Pro	<u>11.46</u>	<u>17.79</u>	<u>21.87</u>
Qwen3.5-397B	8.43	15.68	16.21
GLM-5	7.77	12.65	15.42
DeepSeek-V3.2	7.77	13.44	19.24
GPT-5.4	6.59	16.34	19.10
MiniMax-M2.5	2.77	6.46	7.91

有限。与模拟基准上显著更高的结果 [17, 18] 相比，这进一步说明基于模拟的评测可能高估了真实世界中的意图预测性能。

我们还观察到，模型在 ProCodeBench 上的排名与它们在通用软件工程基准上的常见表现并不一致 [32]。例如，GPT-5.4 的 Pass@1 只有 6.59%，明显落后于 Claude Sonnet 4.6 (13.57%) 和 Gemini 3.1 Pro (11.46%)。这说明主动意图预测需要超出通用推理之外的能力，而如何提升这类能力仍是一个开放问题。

检索增强 LLM 与基于 LLM 的 Agent 结果。 在所有骨干模型上，检索增强 LLM 和基于 LLM 的 Agent 都优于对应的纯 LLM，这说明仅依赖 IDE 交互轨迹通常不足以完成主动意图预测。例如，当使用 GLM-5 作为骨干时，引入仓库上下文后，Pass@1 从 7.77% 提升到了 9.49%–16.73%。在 GPT-5.4、DeepSeek-V3.2 和 Qwen3.5 上也能观察到类似收益。

这种提升在基于 LLM 的 Agent 上尤为明显。与检索增强 LLM 相比，LLM Agent 在四个骨干模型上都取得了最高的 Pass@1。例如，A-RAG 在 GLM-5 上达到 16.73%，在 GPT-5.4 上达到 35.57%，在 DeepSeek-V3.2 上达到 15.81%，在 Qwen3.5 上达到 15.02%。这表明，自主的多轮工具使用对于主动意图预测十分重要。

不过，这种性能增益伴随着显著的效率代价。基于 LLM 的 Agent 平均每次预测需要 23 次工具交互，从而带来更高的计算开销和更长的响应延迟。如何设计同时兼顾更强性能与更高效的方法，仍是未来主动式代码辅助的重要方向。

关键发现 ②： 现有模型在真实世界基准上的表现依然较差，这说明基于模拟的基准可能高估了它们的意图预测能力。仓库级代码上下文能够稳定提升性能，但如何高效获取有用上下文仍是关键挑战。

C. RQ3: 训练研究

为了回答 RQ3，我们在三个开源骨干模型 Qwen-3-8B、GLM-4-9B 和 LLaMA3-8B 上比较了三种训练方案。这三种方案分别是：仅在 LLM 模拟数据上微调 (+Sim.)、仅在真实数据上微调 (+Real)，以及先用模拟数据再用真实数据微调的混合方案 (+Sim.→Real)。所有方案都在同一真实测试集上以 Pass@1 进行评估，同时我们还跟踪训练损失，以分析不同训练方案的优化行为。结果见 Table V 和 Figure 9。**单一数据源训练。** 如 Table V 所示，真实数据对模型训练带来了明确收益，而仅使用 LLM 模拟数据并不能有效迁移到真实测试集。在真实数据上微调 (+Real) 能够稳定提升三个骨干模型的表现：Qwen-3-8B 的 Pass@1 从 2.53% 提升到 5.84%，GLM-4-9B 从 2.24% 提升到 4.93%，LLaMA3-8B

TABLE IV: ProCodeBench (759 个样本) 上检索增强 LLM 与基于 LLM 的 Agent 的 Pass@1 准确率 (%). **红色**表示每行最佳, **蓝色下划线**表示每行次优。

Model	Retrieval-Augmented LLMs				LLM-based Agents	
	RepoCoder	CodeRAG	GraphCoder	RepoGraph	SWE-Agent	A-RAG
GLM-5	9.49	10.01	11.07	10.54	<u>14.23</u>	16.73
GPT-5.4	10.67	11.46	14.36	13.57	<u>32.98</u>	35.57
DeepSeek-V3.2	9.88	10.41	11.86	11.33	<u>12.52</u>	15.81
Qwen3.5	10.14	10.67	11.59	11.07	<u>12.52</u>	15.02

TABLE V: 四种训练方案下的 Pass@1 准确率 (%). **红色**表示该列最佳。

Regime	Qwen-3-8B	GLM-4-9B	LLaMA3-8B
Backbone	2.53	2.24	1.84
+Sim.	1.97	1.65	1.32
+Real	5.84	4.93	3.76
+Sim.→Real	7.63	6.52	5.21

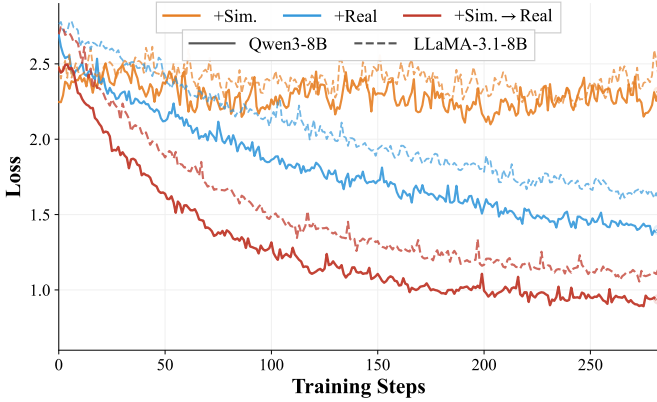


Fig. 9: 不同训练方案在真实数据微调过程中的训练损失曲线。+Sim.→Real 的收敛速度明显快于 +Real, 且最终损失更低。

从 1.84% 提升到 3.76%。相比之下, 仅在 LLM 模拟数据上微调 (+Sim.) 反而使性能低于原始骨干模型, Pass@1 分别降至 1.97%、1.65% 和 1.32%。这说明 LLM 模拟轨迹与真实轨迹之间的落差会直接影响训练效果: 只在模拟数据上训练的模型无法泛化到真实世界的主动意图预测任务。

模拟数据与真实数据的互补性. 尽管 LLM 模拟数据作为单独训练源的效果较差, 但在真实数据微调之前使用它却能发挥作用。混合训练方案 (+Sim.→Real) 在三个骨干模型上都取得了最佳 Pass@1, 分别达到 Qwen-3-8B 的 7.63%、GLM-4-9B 的 6.52% 和 LLaMA3-8B 的 5.21%。与 +Real 相比, 增益分别为 1.79、1.59 和 1.45 个百分点。Figure 9 中的训练损失曲线进一步表明, 以 LLM 模拟数据初始化的模型在真实数据微调阶段收敛更快, 且能达到更低的最终损失。这说明 LLM 模拟数据能够提供一个有用的 warm start, 但真正适应真实开发场景仍离不开真实数据。因此, LLM 模拟数据与真实数据不应被视为可互换的数据源, 而应被视为训练主动式编程助手时的互补信号。

TABLE VI: 不同操作类型对 Pass@1 (%) 的影响。每一行表示移除一种操作类型 (**X**)。↓ 表示相较完整 IDE 交互轨迹的性能下降, ↑ 表示性能提升。

Setting	Claude 4.6	Gemini 3.1	MiniMax
Full sequence	13.57	11.46	2.77
<i>Navigation & Selection</i>			
X cursor_sel.	12.78 ↓0.79	11.20 ↓0.26	3.16 ↑0.39
X copy/paste	13.04 ↓0.53	11.20 ↓0.26	2.90 ↑0.13
X view switching	12.52 ↓1.05	10.67 ↓0.79	3.03 ↑0.26
<i>Execution & Editing</i>			
X terminal	11.86 ↓1.71	9.88 ↓1.58	2.37 ↓0.40
X edit	4.22 ↓9.35	3.16 ↓8.30	0.66 ↓2.11
<i>AI Interaction</i>			
X agent_req.	13.31 ↓0.26	11.33 ↓0.13	2.64 ↓0.13

关键发现 ③: LLM 模拟数据不能替代真实数据, 但两者能够相互补充, 其组合带来的性能提升超过任一单独数据源。

VI. 讨论

A. 消融研究

为了理解不同操作类型对主动意图预测的贡献, 我们进行了操作类型消融研究。具体做法是从完整 IDE 交互轨迹出发, 每次去掉一种操作类型, 并测量 Pass@1 的变化。为了考察不同模型是否以不同方式利用这些信号, 我们选取了三个性能层次不同的模型: Claude Sonnet 4.6、Gemini 3.1 Pro 和 MiniMax-M2.5。结果见 Table VI。

不同操作类型的比较. 如 Table VI 所示, 移除 edit 会在三个模型上造成最大的性能下降, Pass@1 分别下降 9.35、8.30 和 2.11 个百分点。移除 terminal_execution 也会带来明显退化。这些结果表明, 代码编辑与执行反馈为预测开发者意图提供了最直接的信息。相比之下, 移除 agent_request 的影响较小, 可能是因为这种操作类型在数据中出现频率更低。总体而言, 当前模型高度依赖编辑与执行信号, 而真实 IDE 轨迹中的许多其他行为信号仍未被充分利用。

不同模型的敏感性. 能力不同的模型对操作类型移除表现出不同敏感性。Claude Sonnet 4.6 在每一种消融设置下都会退化, 说明它能够利用更广泛范围的 IDE 信号。尤其是导航和选择类操作, 会为更强模型提供互补上下文。相比之下, 当移除 cursor_selection、copy/paste 或 view switching 时, MiniMax-M2.5 的表现反而略有提升, 这说明能力较弱的模型可能会把这些信号当作噪声, 而不是有用信息。

关键发现 ④: 当前模型高度依赖显式执行信息, 而更强模型还能利用来自其他操作的上下文信号。这说明如何有效利用开发者 IDE 交互轨迹, 仍然是主动意图预测中的关键挑战。

B. 效度威胁

意图标注的局限性. 真实 IDE 交互轨迹中的开发者意图是隐含的, 无法被直接观察。因此, 我们的标注流水线需要依赖 LLM 以及诸如显著代码编辑、显式 AI 助手请求等可观察行为信号来推断候选意图。这个过程可能会遗漏一些实际上对主动式辅助很有价值的意图。为缓解这一威胁, 领域专家会在流水线的最后阶段独立复核候选意图, 修正不准确的描述, 并恢复那些被错误过滤掉的有效候选。不过, 从 IDE 交互轨迹中准确恢复开发者意图本身仍是一个开放挑战。

数据采集范围. 受限真实 IDE 交互轨迹采集的成本与隐私风险, 我们不可能穷尽所有开发时间尺度与开发者群体。为缓解这一问题, 我们招募了 1,246 名有经验的工业界开发者, 覆盖前端、后端、全栈、数据库和算法工程五类主要开发场景, 并在他们各自真实维护的项目中采集轨迹。这一设计为真实世界日常开发行为提供了较广覆盖。

出于隐私的受限发布. ProCodeBench 中的 IDE 交互轨迹可能包含敏感的商业信息, 例如企业内部专有代码库。虽然我们通过数据采集协议获得了志愿者授权, 但该协议并不允许无限制公开发布所收集的数据。因此, 我们无法完全开源原始数据集。为了支持可复现性, 我们改为提供一个面向学术研究的受控访问评测平台。研究者可以提交机构信息申请访问, 获批后即可提交自己的模型或 Agent 系统, 并获得在 ProCodeBench 上的评测结果。

VII. 结论

本文研究了 LLM 生成的模拟数据是否能够在真实开发场景中可靠地支撑主动式代码辅助。为分析这一问题, 我们首先通过 VS Code 扩展从 1,246 名志愿者处收集了大规模真实 IDE 交互轨迹, 并为每条真实轨迹构造一条配对的 LLM 模拟轨迹。这种成对设计使我们能够直接考察开发者行为中从模拟到现实的差距。分析表明, LLM 模拟轨迹在行为多样性、时间结构和探索性操作等方面都与真实轨迹存在显著差异, 这说明单靠模拟并不能完整刻画开发者的真实工作方式。

在这些真实数据基础上, 我们构建了面向真实开发场景中主动意图预测的标准化基准 ProCodeBench。针对 LLM、检索增强 LLM 和基于 LLM 的 Agent 的实验表明, 现有模型在该基准上仍然表现吃力, 这说明基于模拟的评测可能高估了意图预测能力。我们进一步研究了真实数据与 LLM 模拟数据对训练的影响。结果表明, 仅依赖 LLM 模拟数据无法很好迁移到真实开发场景, 但它可以在真实数据训练之前提供有用的初始化。这些发现表明, 未来的主动式编程助手应当在真实开发者行为上进行评测, 并学会利用 IDE 交互轨迹中丰富但带噪的信号。

致谢

REFERENCES

- [1] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [2] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, *et al.*, “Competition-level code generation with alphacode,” *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
- [3] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, “An empirical evaluation of using large language models for automated unit test generation,” *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 85–105, 2023.
- [4] A. Sergeyuk, E. Huang, D. Karaeva, A. Serova, Y. Golubev, and I. Ahmed, “Evolving with ai: A longitudinal analysis of developer logs,” *arXiv preprint arXiv:2601.10258*, 2026.
- [5] B. Puryear and G. Sprint, “Github copilot in the classroom: learning to code with ai assistance,” *Journal of Computing Sciences in Colleges*, vol. 38, no. 1, pp. 37–47, 2022.
- [6] S. Barke, M. B. James, and N. Polikarpova, “Grounded copilot: How programmers interact with code-generating models,” *Proceedings of the ACM on Programming Languages*, vol. 7, no. OOPSLA1, pp. 85–111, 2023.
- [7] J. T. Liang, C. Yang, and B. A. Myers, “A large-scale survey on the usability of ai programming assistants: Successes and challenges,” in *Proceedings of the 46th IEEE/ACM international conference on software engineering*, pp. 1–13, 2024.
- [8] R. Khojah, M. Mohamad, P. Leitner, and F. G. de Oliveira Neto, “Beyond code generation: An observational study of chatgpt usage in software engineering practice,” *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1819–1840, 2024.
- [9] M. Bruch, M. Monperrus, and M. Mezini, “Learning from examples to improve code completion systems,” in *Proceedings of the 7th joint meeting of the European software engineering conference and the ACM SIGSOFT symposium on the foundations of software engineering*, pp. 213–222, 2009.
- [10] V. Raychev, M. Vechev, and E. Yahav, “Code completion with statistical language models,” in *Proceedings of the 35th ACM SIGPLAN conference on programming language design and implementation*, pp. 419–428, 2014.
- [11] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. R. Narasimhan, and O. Press, “SWE-agent: Agent-computer interfaces enable automated software engineering,” in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [12] V. Chen, A. Zhu, S. Zhao, H. Mozannar, D. Sontag, and A. Talwalkar, “Need help? designing proactive ai assistants for programming,” in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pp. 1–18, 2025.
- [13] S. Zhao, A. Zhu, H. Mozannar, D. Sontag, A. Talwalkar, and V. Chen, “Codinggenie: A proactive

- llm-powered programming assistant,” in *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, pp. 1168–1172, 2025.
- [14] N. Tang, C. Chen, Z. Fang, G. Xu, M. Dhakal, Y. Shi, C. McMillan, Y. Huang, and T. J.-J. Li, “Programming by chat: A large-scale behavioral analysis of 11,579 real-world ai-assisted ide sessions,” *arXiv preprint arXiv:2604.00436*, 2026.
 - [15] N. Kuo, A. Sergeyuk, V. Chen, and M. Izadi, “Developer interaction patterns with proactive ai: A five-day field study,” *arXiv preprint arXiv:2601.10253*, 2026.
 - [16] H. Mozannar, G. Bansal, A. Fournay, and E. Horvitz, “Reading between the lines: Modeling user behavior and costs in ai-assisted programming,” in *Proceedings of the 2024 CHI conference on human factors in computing systems*, pp. 1–16, 2024.
 - [17] Y. Lu, S. Yang, C. Qian, G. Chen, Q. Luo, Y. Wu, H. Wang, X. Cong, Z. Zhang, Y. Lin, *et al.*, “Proactive agent: Shifting llm agents from reactive responses to active assistance,” *arXiv preprint arXiv:2410.12361*, 2024.
 - [18] J. Kim, J. Choi, W. Chay, D. Kyung, Y. Kwon, Y. Jo, and E. Choi, “Propersim: Developing proactive and personalized ai assistants through user-assistant simulation,” *arXiv preprint arXiv:2509.21730*, 2025.
 - [19] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, “Repocoder: Repository-level code completion through iterative retrieval and generation,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 2471–2484, 2023.
 - [20] S. Ouyang, W. Yu, K. Ma, Z. Xiao, Z. Zhang, M. Jia, J. Han, H. Zhang, and D. Yu, “Repograph: Enhancing ai software engineering with repository-level code graph,” in *13th International Conference on Learning Representations, ICLR 2025*, pp. 30361–30384, International Conference on Learning Representations, ICLR, 2025.
 - [21] M. Du, B. Xu, C. Zhu, S. Wang, P. Wang, X. Wang, and Z. Mao, “A-rag: Scaling agentic retrieval-augmented generation via hierarchical retrieval interfaces,” *arXiv preprint arXiv:2602.03442*, 2026.
 - [22] J. Wang and Y. Chen, “A review on code generation with llms: Application and evaluation,” in *2023 IEEE International Conference on Medical Artificial Intelligence (MedAI)*, pp. 284–289, IEEE, 2023.
 - [23] C. S. Xia, Y. Wei, and L. Zhang, “Automated program repair in the era of large pre-trained language models,” in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pp. 1482–1494, IEEE, 2023.
 - [24] Y. Ding, Z. Wang, W. U. Ahmad, H. Ding, M. Tan, N. Jain, M. K. Ramanathan, R. Nallapati, P. Bhatia, D. Roth, and B. Xiang, “CrossCodeEval: A diverse and multilingual benchmark for cross-file code completion,” in *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023.
 - [25] Y. Deng, W. Lei, W. Lam, and T.-S. Chua, “A survey on proactive dialogue systems: Problems, methods, and prospects,” *arXiv preprint arXiv:2305.02750*, 2023.
 - [26] X. Zhou, W. Sun, Q. Ma, Y. Xie, J. Liu, W. Du, S. Welleck, Y. Yang, G. Neubig, S. T. Wu, *et al.*, “Mind the sim2real gap in user simulation for agentic tasks,” *arXiv preprint arXiv:2603.11245*, 2026.
 - [27] Y. Tang, H. Tang, T. Cao, L. Nguyen, A. Zhang, X. Cao, C. Liu, W. Ding, and Y. Li, “ProAgentBench: Evaluating llm agents for proactive assistance with real-world data,” *arXiv preprint arXiv:2602.04482*, 2025.
 - [28] Y. Chai, S. Tang, H. Xiao, R. Liu, and H. Li, “Pirabench: A transition from reactive gui agents to gui-based proactive intent recommendation agents,” *arXiv preprint arXiv:2603.08013*, 2026.
 - [29] J. Li, G. Li, Y. Zhao, Y. Li, H. Liu, H. Zhu, L. Wang, K. Liu, Z. Fang, L. Wang, *et al.*, “Deveval: A manually-annotated code generation benchmark aligned with real-world code repositories,” in *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 3603–3614, 2024.
 - [30] J. Li, G. Li, X. Zhang, Y. Zhao, Y. Dong, Z. Jin, B. Li, F. Huang, and Y. Li, “Evocodebench: An evolving code generation benchmark with domain-specific evaluations,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 57619–57641, 2024.
 - [31] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. J. Cai, M. Terry, Q. V. Le, and C. Sutton, “Program synthesis with large language models,” *arXiv preprint arXiv:2108.07732*, 2021.
 - [32] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, “SWE-bench: Can language models resolve real-world github issues?,” in *The Twelfth International Conference on Learning Representations*, 2024.
 - [33] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, “CodeSearchNet challenge: Evaluating the state of semantic code search,” *arXiv preprint arXiv:1909.09436*, 2019.
 - [34] B. Li, W. Wu, Z. Tang, L. Shi, J. Yang, J. Li, S. Yao, C. Qian, B. Hui, Q. Zhang, *et al.*, “DevBench: A comprehensive benchmark for software development,” *arXiv preprint arXiv:2403.08604*, 2024.
 - [35] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, *et al.*, “Judging llm-as-a-judge with mt-bench and chatbot arena,” *Advances in neural information processing systems*, vol. 36, pp. 46595–46623, 2023.
 - [36] P. Wang, L. Li, L. Chen, Z. Cai, D. Zhu, B. Lin, Y. Cao, L. Kong, Q. Liu, T. Liu, *et al.*, “Large language models

are not fair evaluators,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 9440–9450, 2024.

- [37] GLM-5-Team, A. Zeng, X. Lv, Z. Hou, Z. Du, Q. Zheng, B. Chen, D. Yin, C. Ge, C. Huang, C. Xie, C. Zhu, C. Yin, C. Wang, G. Pan, H. Zeng, H. Zhang, H. Wang, H. Chen, J. Zhang, J. Jiao, J. Guo, J. Wang, J. Du, J. Wu, K. Wang, L. Li, L. Fan, L. Zhong, M. Liu, M. Zhao, P. Du, Q. Dong, R. Lu, Shuang-Li, S. Cao, S. Liu, T. Jiang, X. Chen, X. Zhang, X. Huang, X. Dong, Y. Xu, Y. Wei, Y. An, Y. Niu, Y. Zhu, Y. Wen, Y. Cen, Y. Bai, Z. Qiao, Z. Wang, Z. Wang, Z. Zhu, Z. Liu, Z. Li, B. Wang, B. Wen, C. Huang, C. Cai, C. Yu, C. Li, C. Hu, C. Zhang, D. Zhang, D. Lin, D. Yang, D. Wang, D. Ai, E. Zhu, F. Yi, F. Chen, G. Wen, H. Sun, H. Zhao, H. Hu, H. Zhang, H. Liu, H. Zhang, H. Peng, H. Tai, H. Zhang, H. Liu, H. Wang, H. Yan, H. Ge, H. Liu, H. Chu, J. Zhao, J. Wang, J. Zhao, J. Ren, J. Wang, J. Zhang, J. Gui, J. Zhao, J. Li, J. An, J. Li, J. Yuan, J. Du, J. Liu, J. Zhi, J. Duan, K. Zhou, K. Wei, K. Wang, K. Luo, L. Zhang, L. Sha, L. Xu, L. Wu, L. Ding, L. Chen, M. Li, N. Lin, P. Ta, Q. Zou, R. Song, R. Yang, S. Tu, S. Yang, S. Wu, S. Zhang, S. Li, S. Li, S. Fan, W. Qin, W. Tian, W. Zhang, W. Yu, W. Liang, X. Kuang, X. Cheng, X. Li, X. Yan, X. Hu, X. Ling, X. Fan, X. Xia, X. Zhang, X. Zhang, X. Pan, X. Zou, X. Zhang, Y. Liu, Y. Wu, Y. Li, Y. Wang, Y. Zhu, Y. Tan, Y. Zhou, Y. Pan, Y. Zhang, Y. Su, Y. Geng, Y. Yan, Y. Tan, Y. Bi, Y. Shen, Y. Yang, Y. Li, Y. Liu, Y. Wang, Y. Li, Y. Wu, Y. Zhang, Y. Duan, Y. Zhang, Z. Liu, Z. Jiang, Z. Yan, Z. Zhang, Z. Wei, Z. Chen, Z. Feng, Z. Yao, Z. Chai, Z. Wang, Z. Zhang, B. Xu, M. Huang, H. Wang, J. Li, Y. Dong, and J. Tang, “Glm-5: from vibe coding to agentic engineering,” 2026.
- [38] Qwen Team, “Qwen3.5: Towards native multimodal agents,” February 2026.
- [39] S. Zhang, Y. Ding, S. Lian, S. Song, and H. Li, “Coderag: Finding relevant and necessary knowledge for retrieval-augmented repository-level code completion,” in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 23289–23299, 2025.
- [40] W. Liu, A. Yu, D. Zan, B. Shen, W. Zhang, H. Zhao, Z. Jin, and Q. Wang, “Graphcoder: Enhancing repository-level code completion via coarse-to-fine retrieval based on code context graph,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pp. 570–581, 2024.
- [41] Q. Team, “Qwen3 technical report,” 2025.
- [42] Team GLM, A. Zeng, B. Xu, B. Wang, C. Zhang, D. Yin, D. Rojas, G. Feng, H. Zhao, H. Lai, H. Yu, H. Wang, J. Sun, J. Zhang, J. Cheng, J. Gui, J. Tang, J. Zhang, J. Li, L. Zhao, L. Wu, L. Zhong, M. Liu, M. Huang, P. Zhang, Q. Zheng, R. Lu, S. Duan, S. Zhang, S. Cao, S. Yang, W. L. Tam, W. Zhao, X. Liu, X. Xia, X. Zhang, X. Gu, X. Lv, X. Liu, X. Liu, X. Yang, X. Song, X. Zhang, Y. An, Y. Xu, Y. Niu, Y. Yang, Y. Li, Y. Bai, Y. Dong, Z. Qi, Z. Wang, Z. Yang, Z. Du, Z. Hou, and Z. Wang, “Chatglm: A family of large language models from glm-130b to glm-4 all tools,” 2024.
- [43] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, *et al.*, “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.